

Relatório de Laboratório

Lab01 — Características de Repositórios Populares

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab01 — Características de Repositórios Populares
Grupo (trio)	Islayder Jackson · Fernanda Soares · Vinicius Gomes
Link do repositório / GitHub Projects	Repositório: https://github.com/fesoareess/Laboratorio-Medicao-e-experimentacao GitHub Projects: https://github.com/fesoareess/Laboratorio-Medicao-e-experimentacao/projects
Data de entrega	26/08/2026

1. Introdução

Repositórios open source populares reúnem projetos com trajetórias, tecnologias e práticas de manutenção distintas. A caracterização sistemática desses projetos por métricas observáveis de Engenharia de Software permite discutir maturidade, atividade colaborativa, publicação de versões, recência de atualização, linguagens e tratamento de issues sem depender apenas de percepções anedóticas.

O Lab01 investiga os 1.000 repositórios mais populares do GitHub, operacionalizando popularidade pelo número de estrelas. A amostra foi minerada pela API GraphQL e analisada por meio das Questões de Pesquisa oficiais RQ01–RQ06:

- RQ01 — Sistemas populares são maduros/antigos?
- RQ02 — Sistemas populares recebem muita contribuição?
- RQ03 — Sistemas populares lançam releases com frequência?
- RQ04 — Sistemas populares são atualizados com frequência?
- RQ05 — Sistemas populares são escritos nas linguagens mais populares?
- RQ06 — Sistemas populares possuem alto percentual de issues fechadas?

Separadamente, o grupo analisou a RQ07 — Bônus: relação entre popularidade da linguagem, contribuição externa, releases e frequência de atualização.

Hipóteses informais

As hipóteses a seguir foram registradas no template e nos relatórios de Sprint antes da consolidação final dos resultados. Para a RQ01, esperava-se idade relativamente elevada entre repositórios populares; para a RQ02, esperava-se quantidade elevada de pull requests aceitas como indício de atividade colaborativa.

Para a RQ03, esperava-se quantidade relevante de releases, com variação conforme a estratégia de versionamento. Para a RQ04, esperava-se que grande parte dos repositórios apresentasse atualizações recentes. Para a RQ05, esperava-se concentração em tecnologias bem posicionadas na referência de popularidade adotada. Para a RQ06, esperava-se proporção elevada de issues fechadas.

Para a RQ07 bônus, a hipótese informal era que repositórios associados a linguagens mais populares apresentariam, em mediana, maior número de PRs merged, maior total de releases e menor tempo desde o último push. A expectativa foi tratada como ponto de partida descritivo, sem pressupor causalidade.

Como contribuições adicionais, o grupo integrou a RQ07 bônus e uma análise metodológica de robustez/sensibilidade das RQ01/RQ02 a valores extremos segundo o critério IQR.

2. Contexto

Este estudo corresponde ao Lab01 da disciplina Laboratório de Experimentação de Software. Seu objeto são os 1.000 repositórios com maior número de estrelas recuperados pela busca do GitHub no momento da coleta. As estrelas são usadas como proxy operacional de popularidade, enquanto as demais características são obtidas dos campos disponibilizados pela plataforma.

O desenvolvimento ocorreu incrementalmente em três Sprints: preparação da arquitetura e das métricas; expansão e validação da coleta para 1.000 itens; e análise, visualização, contribuições complementares e consolidação do relatório. O grupo registrou o TIOBE Index de agosto de 2026 como referência externa de popularidade de linguagens e manteve essa fonte nas RQ05 e RQ07.

3. Metodologia

A metodologia combina seleção da amostra, coleta paginada, normalização, persistência incremental, validação, análise descritiva e visualização. Cada linha do CSV final representa um repositório. A análise principal mantém os 1.000 registros; a RQ07 reutiliza métricas já coletadas, e a visão sem extremos IQR é usada exclusivamente como verificação complementar de sensibilidade nas RQ01/RQ02.

3.1 Principais Desafios

A query integrada reúne os campos necessários às seis RQs oficiais; a RQ07 deriva comparações das métricas já coletadas e não exige nova consulta. Durante a evolução da coleta, páginas com 100 repositórios produziram erro HTTP 502, o que exigiu reduzir o volume por chamada e priorizar estabilidade. A escala de 1.000 repositórios também tornou necessárias paginação automática, requisições sequenciais e acompanhamento do rate limit.

Interrupções e falhas temporárias poderiam deixar uma coleta longa incompleta. Para reduzir esse risco, o projeto passou a persistir cada página em CSV e a manter checkpoint com cursor, quantidade, página e informações de rate limit, permitindo retomada sem reiniciar todo o processo e evitando duplicações.

Também foi necessário distinguir ausência de informação de valores numéricos legítimos: linguagem nula foi registrada como “Não informado”, e a proporção de issues fechadas foi mantida indefinida quando o total de issues era zero. Na análise, distribuições muito assimétricas — especialmente a de PRs merged — exigiram mediana, quartis, boxplots e uma visualização logarítmica complementar.

3.2 Tomadas de Decisão

- Uso da API GraphQL para solicitar, em uma mesma consulta, os campos necessários às RQ01–RQ06.
- Busca stars:>0 sort:stars-desc e limite de 1.000 repositórios para operacionalizar a amostra por estrelas.
- PAGE_SIZE = 10 e coleta sequencial, priorizando confiabilidade após a ocorrência de HTTP 502 com páginas maiores.
- Retry com backoff limitado para falhas transitórias; erros permanentes de autenticação ou GraphQL encerram a execução.
- Persistência incremental em CSV e checkpoint/resume para recuperação e prevenção de duplicações.

- Uso de média, mediana, quartis, IQR e assimetria para descrever distribuições; transformação log1p apenas na visualização da RQ02.
- Manutenção de todos os valores extremos na amostra oficial; a exclusão IQR aparece somente na análise complementar de sensibilidade.
- Política de WIP do GitHub Projects com limite de 3 itens em Doing e 2 em Review, registrada para reduzir multitarefa e acúmulo de revisão.

3.3 Etapas

3.3.1 Desenvolvimento por Sprints

Sprint	Entregas	Responsável(is)	Issues
Sprint 1	Configuração e acompanhamento do GitHub Projects.	Fernanda Soares	#1
Sprint 1	Implementação das métricas RQ03/RQ04.	Fernanda Soares	#2
Sprint 1	Cliente GraphQL e métricas RQ01/RQ02.	Islayder Jackson	#3
Sprint 1	Arquitetura inicial do repositório.	Islayder Jackson	#4
Sprint 1	Linguagem principal RQ05 e razão de issues fechadas RQ06.	Vinicius Gomes	#5
Sprint 2	Coleta robusta de 1.000 itens, CSV, paginação, checkpoint/resume, retry/backoff e validação.	Islayder Jackson	#6
Sprint 2	Validação de RQ03/RQ04 e snapshot do quadro.	Fernanda Soares	#7
Sprint 2	Validação de RQ05/RQ06 e definição do TIOBE.	Vinicius Gomes	#8
Sprint 3	Análise e visualização de RQ01/RQ02; análise de sensibilidade IQR.	Islayder Jackson	#10
Sprint 3	Análise e visualização de RQ03/RQ04.	Fernanda Soares	#11
Sprint 3	Análise e visualização de RQ05/RQ06 e RQ07 bônus.	Vinicius Gomes	#12

3.3.2 Procedimento de Coleta e Validação

A query GraphQL recebe first, after e a expressão de busca. A primeira página é consultada sem cursor; as seguintes reutilizam pageInfo.endCursor enquanto pageInfo.hasNextPage for verdadeiro. Para cada node, são coletados identificação, estrelas e os campos das seis métricas oficiais. A normalização converte datas em dias, contagens em inteiros, linguagem ausente em categoria explícita e a razão de issues em valor entre 0 e 1 ou indefinido quando o denominador é zero. A RQ07 reutiliza essas métricas e agrupa os registros por primaryLanguage.

Cada página validada é anexada ao CSV. O checkpoint armazena cursor, quantidade persistida, estado de conclusão, páginas e rate limit. A validação final verifica quantidade esperada, unicidade, campos obrigatórios, não negatividade das contagens, coerência entre issues totais e fechadas, domínio da razão e ordenação decrescente por estrelas. Testes offline cobrem métricas, paginação, retry, persistência, retomada, análise e robustez sem depender da API.

3.3.3 Configuração do Processo

O trabalho foi acompanhado no GitHub Projects v2 pelo fluxo Backlog → To Do → Doing → Review → Done. Issues e Assignees registram a responsabilidade principal, e os snapshots preservam estados do quadro ao longo das três Sprints. As análises de RQ01/RQ02, RQ03/RQ04 e RQ05/RQ06/RQ07 foram vinculadas às Issues #10, #11 e #12, respectivamente.

3.4 Ferramentas

- Python para cliente HTTP, coleta, normalização, persistência, testes e análise.
- API GraphQL do GitHub como fonte dos dados dos repositórios.
- Pandas para cálculo e estruturação dos dados analisados.
- Matplotlib para figuras estáticas incorporadas ao relatório.
- python-dotenv para carregar GITHUB_TOKEN a partir do ambiente local.
- unittest para testes automatizados offline.
- Git e GitHub para versionamento; GitHub Projects v2 para gestão do trabalho.
- CSV para dados coletados e processados; JSON para checkpoint.

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta/Fonte
RQ01	Idade do repositório	Data de referência – createdAt	Dias	GitHub GraphQL API / script próprio
RQ02	PRs merged	totalCount de pullRequests com estado MERGED; proxy de contribuição aceita, sem identificar autoria externa	Quantidade de PRs	GitHub GraphQL API / script próprio
RQ03	Total de releases	totalCount de releases formais do GitHub	Quantidade	GitHub GraphQL API / script próprio
RQ04	Tempo desde a última atualização	Data de referência – pushedAt	Dias	GitHub GraphQL API / script próprio

RQ	Métrica	Definição Operacional	Unidade	Ferramenta/Fonte
RQ05	Linguagem principal e popularidade	primaryLanguage.name; “Não informado” quando ausente; comparação com TIOBE agosto/2026	Categoria / posição	GitHub GraphQL API / TIOBE
RQ06	Proporção de issues fechadas	closedIssues.totalCount ÷ issues.totalCount; indefinida quando o total é zero	Proporção (0–1)	GitHub GraphQL API / script próprio
RQ07 — Bônus	Relação entre popularidade da linguagem, contribuição externa, releases e frequência de atualização	Agrupamento por primaryLanguage; medianas de PRs merged, total de releases e dias desde o último push; comparação descritiva com o TIOBE Index de agosto de 2026	PRs / releases / dias por linguagem	Dataset da API GraphQL do GitHub / TIOBE

3.6 Inovações Propostas pelo Grupo (30% da nota)

O grupo apresenta duas contribuições complementares: uma análise metodológica de sensibilidade a valores extremos nas RQ01 e RQ02 e a RQ07 bônus, que relaciona as métricas já coletadas à linguagem principal e ao TIOBE Index. Ambas preservam a amostra oficial e têm finalidade descritiva.

3.6.1 Análise de sensibilidade por IQR

Para as métricas numéricas das RQ01 e RQ02, valores extremos foram identificados pela regra clássica do intervalo interquartil: $IQR = Q3 - Q1$; limite inferior = $Q1 - 1,5 \times IQR$; limite superior = $Q3 + 1,5 \times IQR$.

Foram comparadas média, mediana, desvio padrão, mínimo e máximo da distribuição completa com as mesmas medidas na subamostra sem extremos IQR, além das diferenças absolutas e percentuais. Trata-se somente de uma análise de sensibilidade: nenhum registro foi removido do CSV nem das respostas oficiais às RQs, que continuam baseadas nos 1.000 repositórios.

3.6.2 RQ07 bônus por linguagem e TIOBE

A RQ07 — Bônus: relação entre popularidade da linguagem, contribuição externa, releases e frequência de atualização — é uma análise complementar. Seu objetivo é verificar se as medianas de PRs merged, total de GitHub Releases e dias desde o último push diferem entre repositórios agrupados pela linguagem principal e pela presença da linguagem no top 10 do TIOBE Index de agosto de 2026. PRs merged permanece uma proxy operacional de contribuição aceita e não identifica necessariamente autoria externa.

Na comparação agregada, foram usados todos os 913 repositórios com linguagem conhecida: 506 em linguagens do TIOBE top 10 e 407 nas demais; os 87 registros com linguagem “Não informado” foram excluídos apenas dessa comparação. Nos três gráficos individuais por linguagem, aplicou-se o corte mínimo de 15 repositórios para evitar categorias com amostras muito pequenas; esse corte não foi aplicado à comparação agregada. Foram elegíveis 10 linguagens. A comparação é descritiva, sensível à composição dos grupos e não implica causalidade.

4. Resultados

4.1 Coleta de Dados

A coleta final foi concluída com 1.000 registros, 1.000 repositórios únicos e 0 duplicados. O checkpoint registra 100 páginas com PAGE_SIZE = 10 e estado completed = true. A ordenação decrescente por estrelas e os campos obrigatórios foram validados.

O checkpoint final foi atualizado em 16 de agosto de 2026, às 05:34 UTC, com 4.882 unidades restantes informadas pelo rate limit. Não foi localizado no projeto um tempo total de execução documentado, portanto esse valor não é reportado.

Não houve ausência nas métricas RQ01/RQ02. Para RQ05, 87 repositórios foram mantidos na categoria “Não informado”. Para RQ06, 43 repositórios possuíam zero issues e, coerentemente, 43 razões foram mantidas como indefinidas em vez de convertidas para zero. Esses casos não foram removidos.

4.2 Visualização Gráfica

As figuras a seguir apresentam os resultados das Questões de Pesquisa analisadas.

RQ01 — Sistemas populares são maduros/antigos?

A idade apresenta mediana de 2.821 dias e média de 2.793,921 dias. Q1 é 1.280,25 dias e Q3 é 4.142,50 dias; o intervalo observado vai de 2 a 6.701 dias. O histograma mostra a amplitude da distribuição sem indicar uma concentração exclusiva em uma única faixa etária.

Distribuição da idade dos repositórios populares

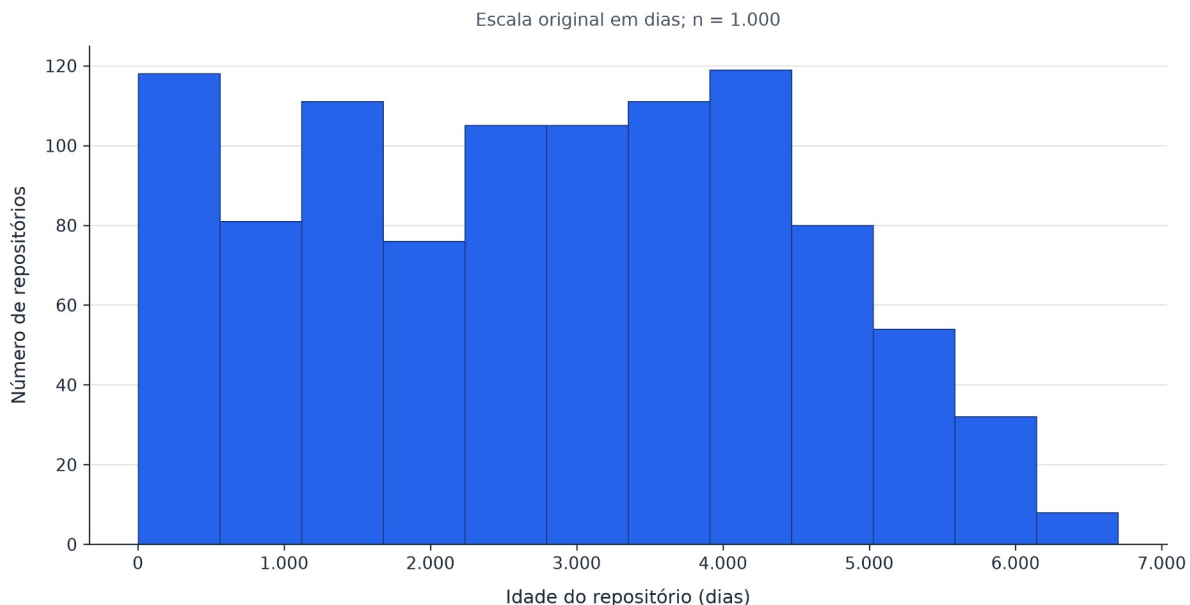


Figura 1 — Distribuição da idade dos 1.000 repositórios populares, em dias. Fonte: elaboração própria a partir do dataset coletado.

RQ02 — Sistemas populares recebem muita contribuição?

A quantidade de PRs merged apresenta mediana de 765,5 e média de 4.210,608. Q1 é 175, Q3 é 3.388,25, o mínimo é 0 e o máximo é 103.110. A assimetria de 5,36 evidencia cauda direita pronunciada.

Para tornar a forma da distribuição legível, a Figura 2 usa $\log_{10}(x)$ somente na visualização. Nenhum valor do dataset foi transformado para o cálculo das estatísticas, que permanecem na escala original de quantidade de PRs merged.

Distribuição de pull requests merged em escala $\log_{10}$

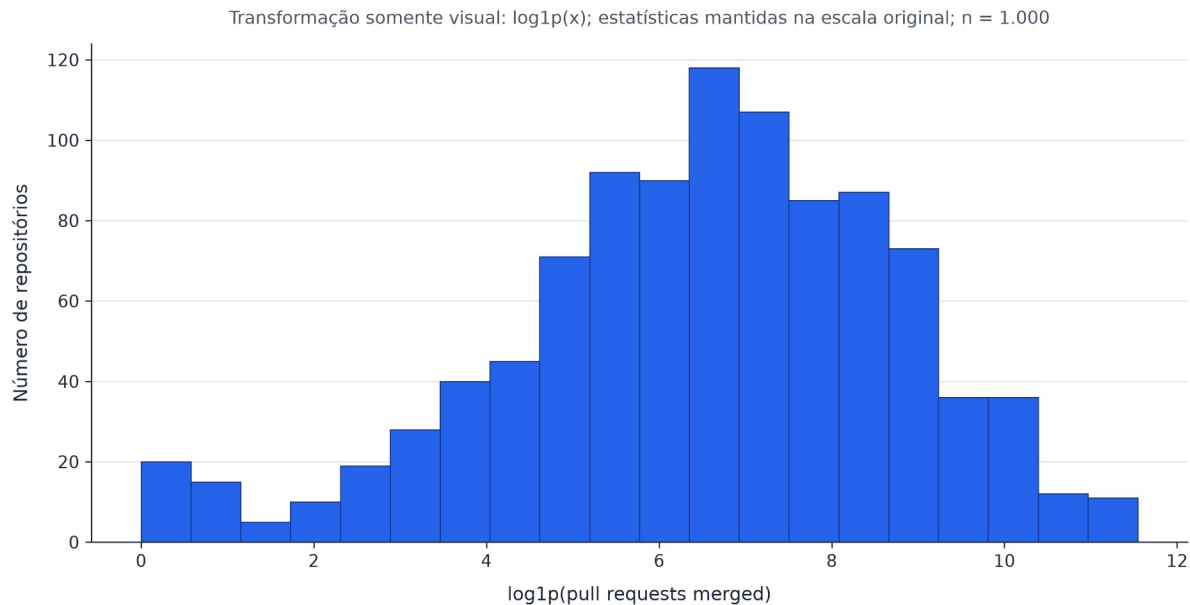


Figura 2 — Distribuição visual de PRs merged após transformação $\log_{10}$. As estatísticas do texto permanecem na escala original.

Fonte: elaboração própria.

RQ03 — Sistemas populares lançam releases com frequência?

O total de GitHub Releases apresenta mediana de 39,5 e média de 127,29 por repositório. Q1 é 0 e Q3 é 148,25; o intervalo observado vai de 0 a 1.000. A assimetria de 2,61 indica concentração em valores baixos e cauda direita pronunciada. Há 280 repositórios (28,0%) sem release formal e 21 repositórios (2,1%) com valor exatamente igual ao máximo observado de 1.000.

A métrica operacional é a quantidade acumulada de releases, e não uma taxa por mês ou ano; assim, a RQ é interpretada em termos do volume observado. A documentação oficial define totalCount como a contagem total de itens da conexão e não documenta um teto de 1.000 para esse campo, motivo pelo qual 1.000 é reportado apenas como máximo observado. A ausência de GitHub Releases formais não implica necessariamente inatividade, pois os projetos podem empregar outros mecanismos de versionamento ou distribuição.

Distribuição do total de releases por repositório

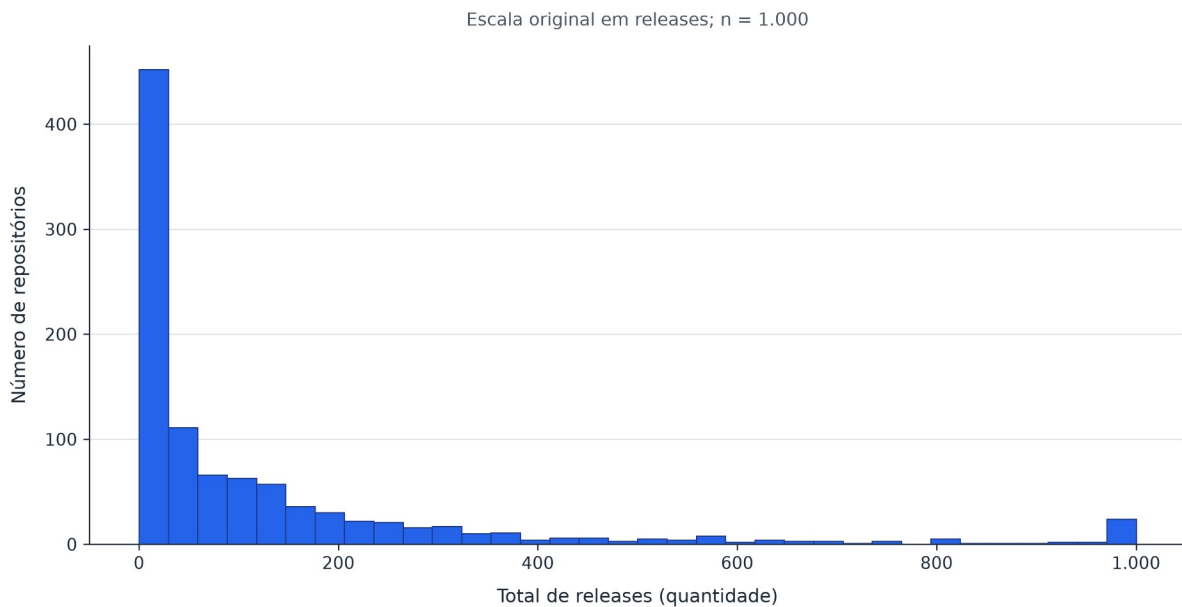


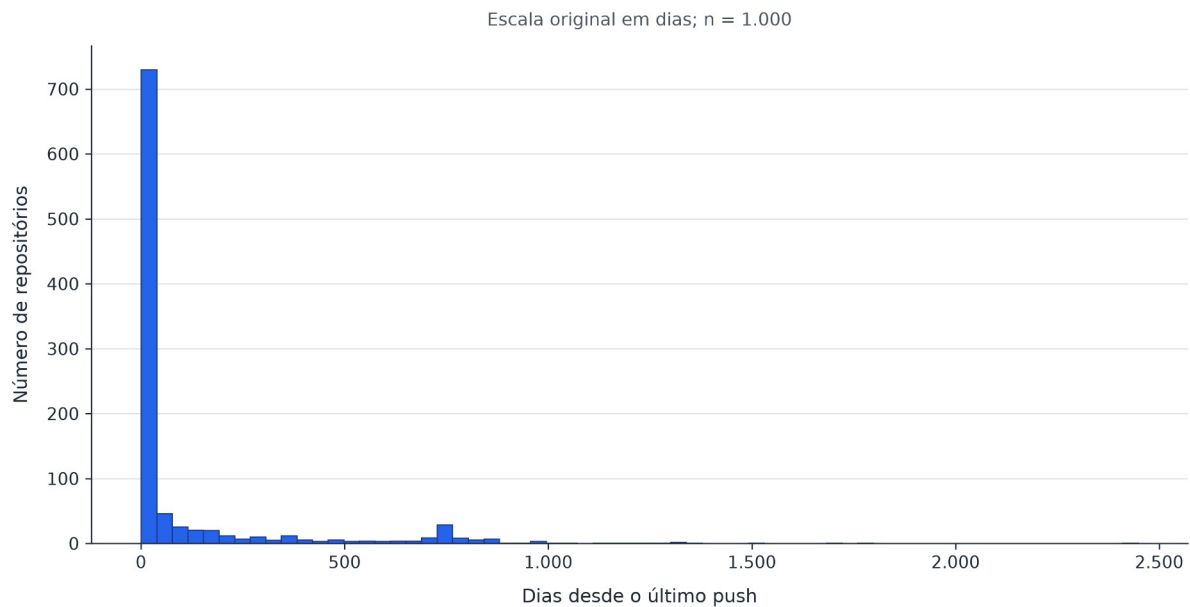
Figura 3 — Distribuição do total de GitHub Releases por repositório. Fonte: elaboração própria a partir do dataset coletado.

RQ04 — Sistemas populares são atualizados com frequência?

O tempo desde o último push apresenta mediana de 3 dias e média de 114,06 dias. Q1 é 1 dia e Q3 é 52 dias; o intervalo observado vai de 0 a 2.448 dias. Há 177 repositórios (17,7%) com push no dia da coleta, 602 (60,2%) com push nos sete dias anteriores à referência e 114 (11,4%) sem push há mais de 365 dias.

A maioria dos repositórios populares da amostra apresentou atualização recente. O campo pushedAt mede o tempo desde o último push e, isoladamente, não comprova manutenção ativa nem sua qualidade. Os valores refletem a data de coleta, em agosto de 2026.

Distribuição do tempo desde a última atualização



RQ05 — Sistemas populares são escritos nas linguagens mais populares?

Foram identificadas 44 categorias de linguagem na amostra, incluindo “Não informado”. As cinco mais frequentes são Python (22,9%), TypeScript (17,4%), JavaScript (11,0%), “Não informado” (8,7%) e Go (7,6%); Python, TypeScript e JavaScript respondem, juntas, por 51,3% dos repositórios. Conforme o TIOBE Index de agosto de 2026, 506 repositórios (50,6% da amostra total) usam linguagens do top 10 adotado pelo grupo.

A distribuição concentra-se em poucas linguagens e também apresenta uma cauda de categorias menos frequentes. TypeScript e Go têm presença expressiva na amostra, embora não integrem o top 10 do TIOBE usado no estudo; a divergência indica que a metodologia do índice e a frequência de linguagens em repositórios estrelados do GitHub operacionalizam noções distintas de popularidade.

Top 15 linguagens primárias — repositórios populares

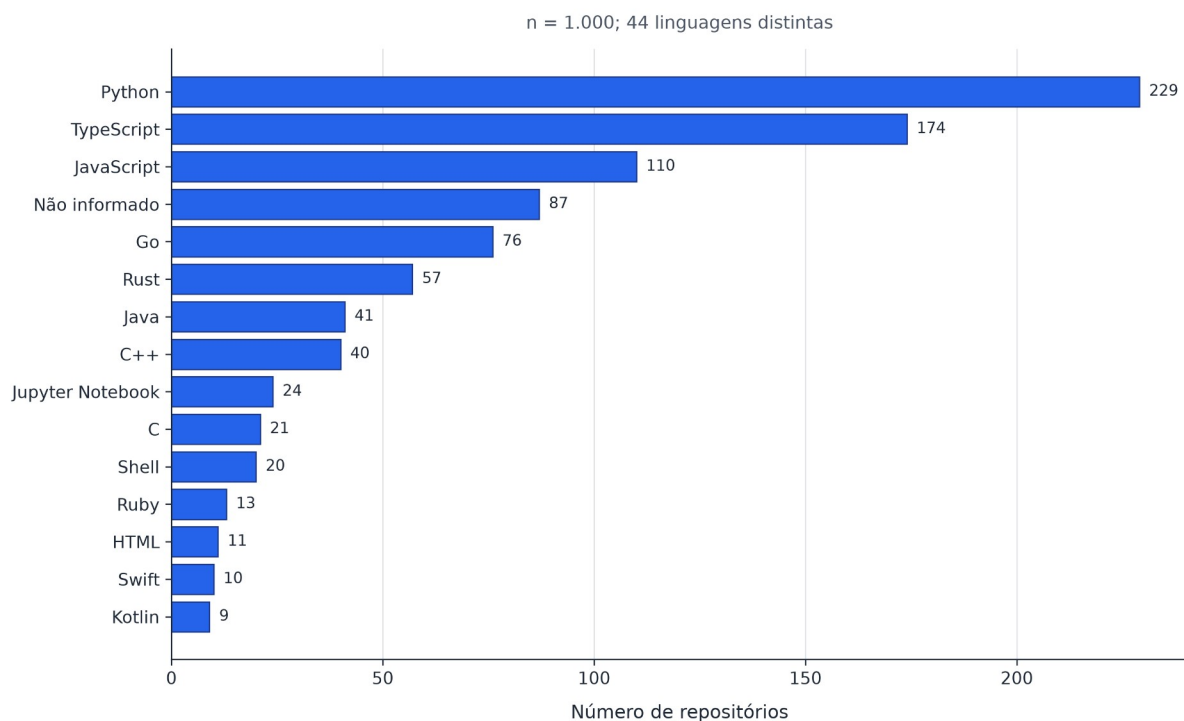


Figura 5 — Top 15 linguagens primárias entre os 1.000 repositórios populares. Fonte: elaboração própria a partir do dataset coletado.

RQ06 — Sistemas populares possuem alto percentual de issues fechadas?

A razão de issues fechadas apresenta mediana de 87,5%, com Q1 em 70,5% e Q3 em 96,8% (IQR de 26,3 pontos percentuais). O mínimo observado foi 7,7% e o máximo, 100%. Dos 1.000 repositórios, 957 têm razão definida; 43 repositórios sem issues foram mantidos como indefinidos e não computados como zero.

Pelo procedimento de validação implementado na Sprint 2, baseado em quartis exclusivos e na regra $1,5 \times \text{IQR}$, foram identificados 39 outliers (4,08% dos 957 valores definidos). A concentração em proporções elevadas descreve o comportamento da amostra, mas não constitui prova isolada da qualidade da manutenção ou da atuação dos mantenedores.

Boxplot da razão de issues fechadas

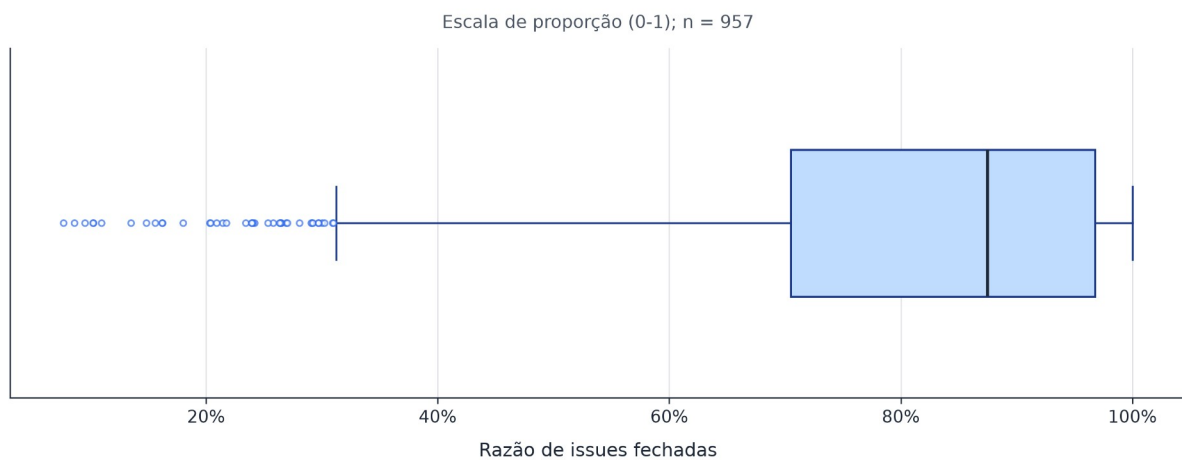


Figura 6 — Boxplot da razão de issues fechadas entre os repositórios com valor definido. Fonte: elaboração própria a partir do dataset coletado.

Distribuição da razão de issues fechadas

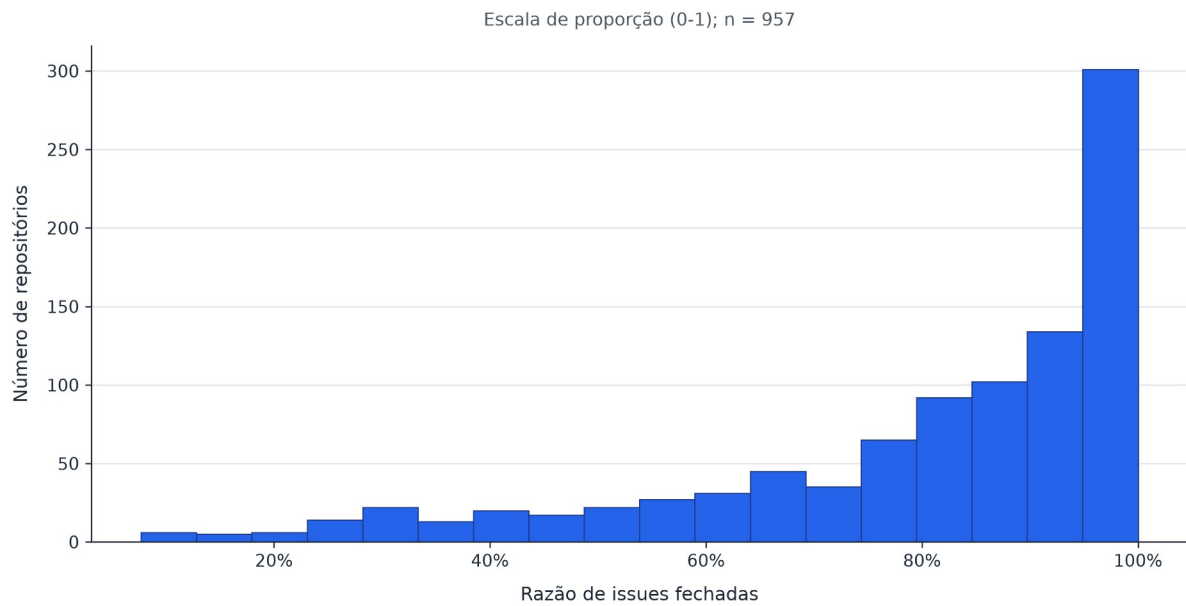


Figura 7 — Distribuição da razão de issues fechadas entre os repositórios com valor definido. Fonte: elaboração própria a partir do dataset coletado.

RQ07 — Bônus: relação entre popularidade da linguagem, contribuição externa, releases e frequência de atualização

A comparação agregada usa todos os 913 repositórios com linguagem conhecida: 506 em linguagens do TIOBE top 10 e 407 nas demais, com exclusão dos 87 registros “Não informado”. As medianas foram, respectivamente: 754,5 e 1.075,0 PRs merged; 40,0 e 83,0 GitHub Releases; e 3,0 e 2,0 dias desde o último push.

Na amostra, o grupo fora do TIOBE top 10 apresentou mediana maior de PRs merged e releases e menor tempo desde o último push. A composição dos grupos ajuda a interpretar o resultado: Python (n = 229) pesa no grupo TIOBE, enquanto TypeScript (n = 174) e Go (n = 76) têm forte presença no grupo das demais linguagens. Os gráficos individuais mostram somente as 10 linguagens com pelo menos 15 repositórios; esse corte não altera a comparação agregada.

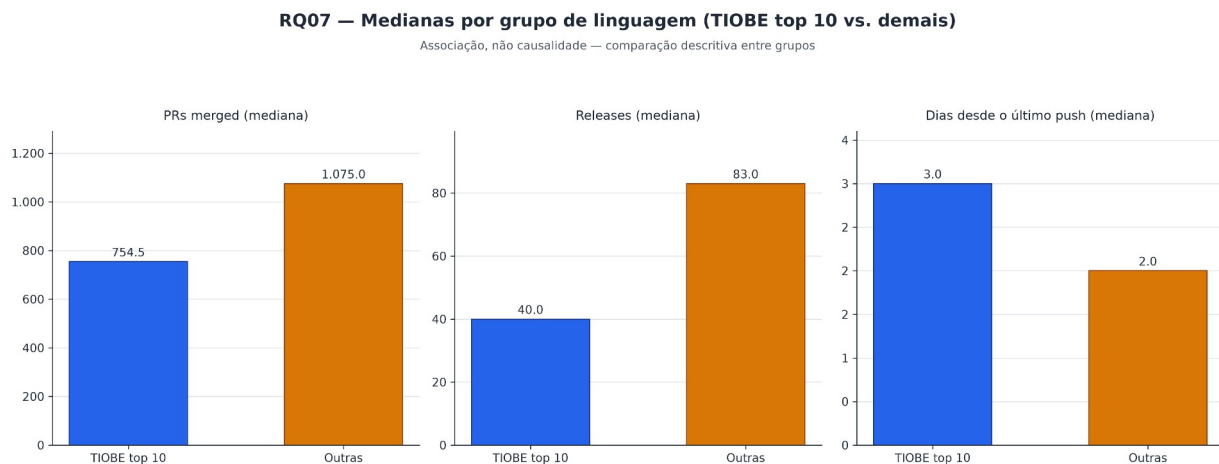


Figura 8 — Medianas de PRs merged, GitHub Releases e dias desde o último push por grupo de linguagem (TIOBE top 10 versus demais). Fonte: elaboração própria a partir do dataset coletado.

RQ07 — PRs merged (mediana) por linguagem

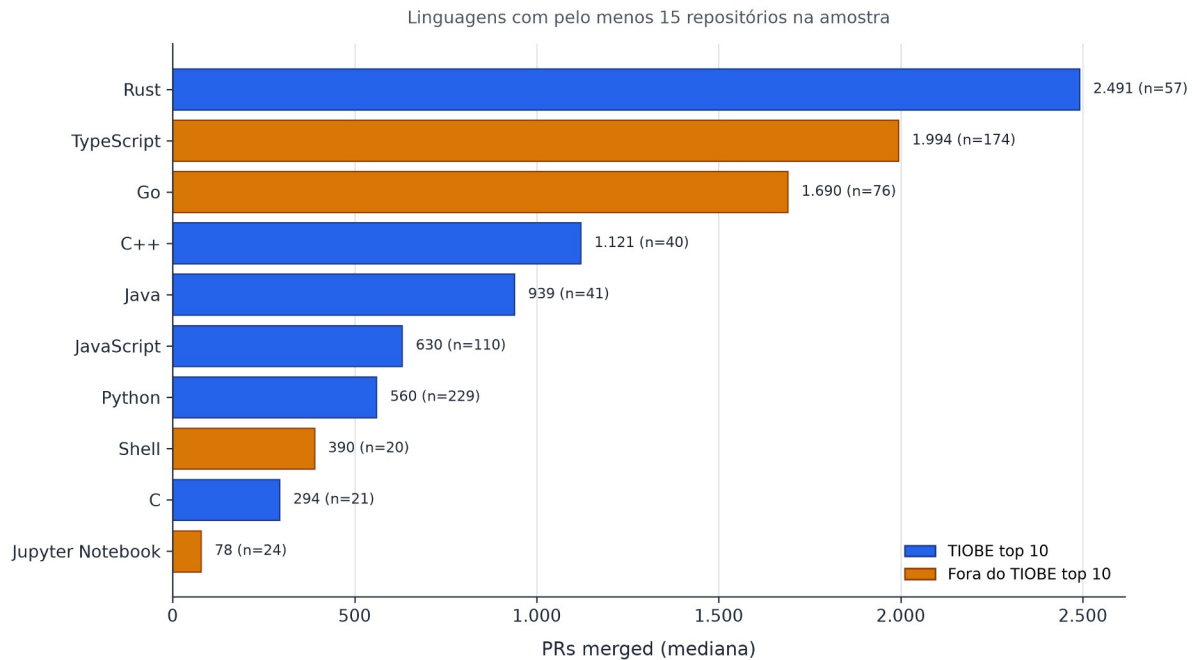


Figura 9 — Mediana de PRs merged por linguagem, considerando categorias com pelo menos 15 repositórios. Fonte: elaboração própria a partir do dataset coletado.

RQ07 — Releases (mediana) por linguagem

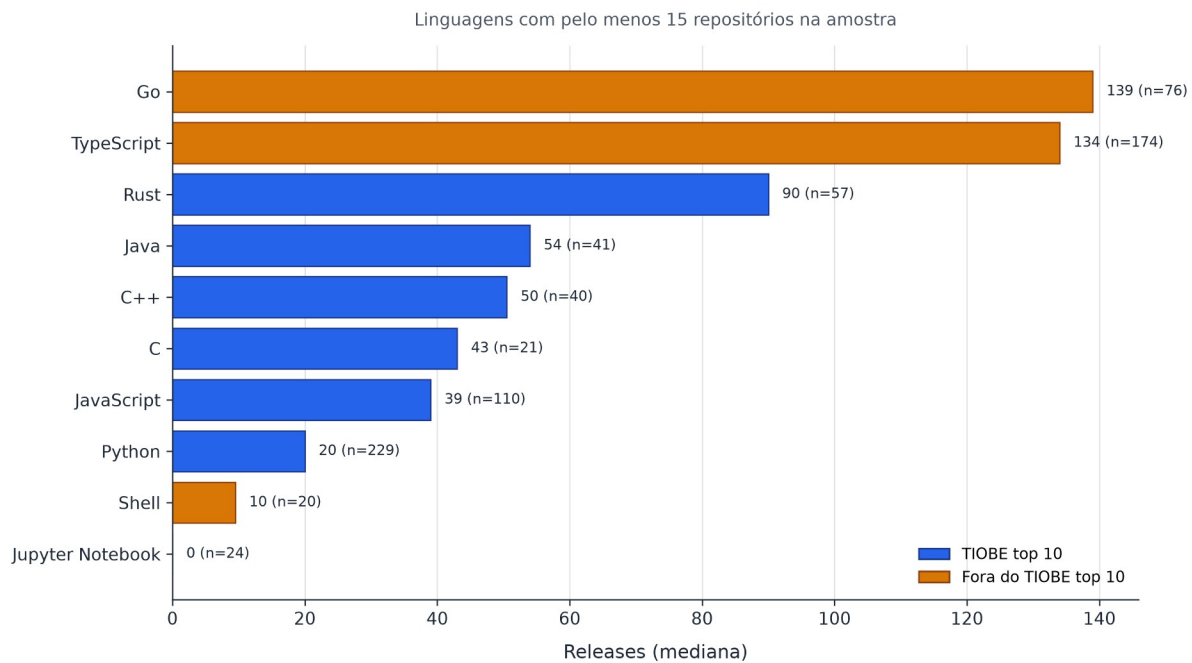


Figura 10 — Mediana do total de GitHub Releases por linguagem, considerando categorias com pelo menos 15 repositórios. Fonte: elaboração própria a partir do dataset coletado.

RQ07 — Dias desde o último push (mediana) por linguagem

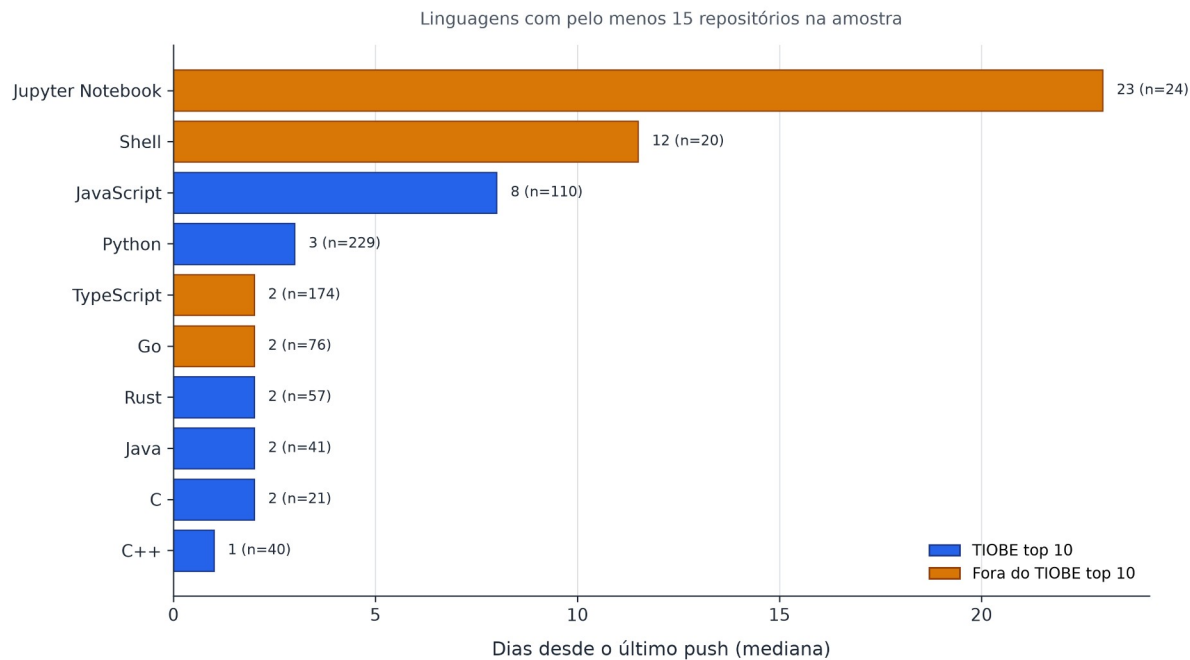


Figura 11 — Mediana dos dias desde o último push por linguagem, considerando categorias com pelo menos 15 repositórios.

Fonte: elaboração própria a partir do dataset coletado.

Visualização da contribuição de robustez

A RQ01 não apresentou extremos pelo critério IQR, portanto média e mediana permaneceram inalteradas. Na RQ02, foram identificados 123 extremos (12,3% dos valores válidos). Na análise complementar, a média variou -67,44% e a mediana, -23,97%.

Sensibilidade das medidas-resumo a valores extremos IQR

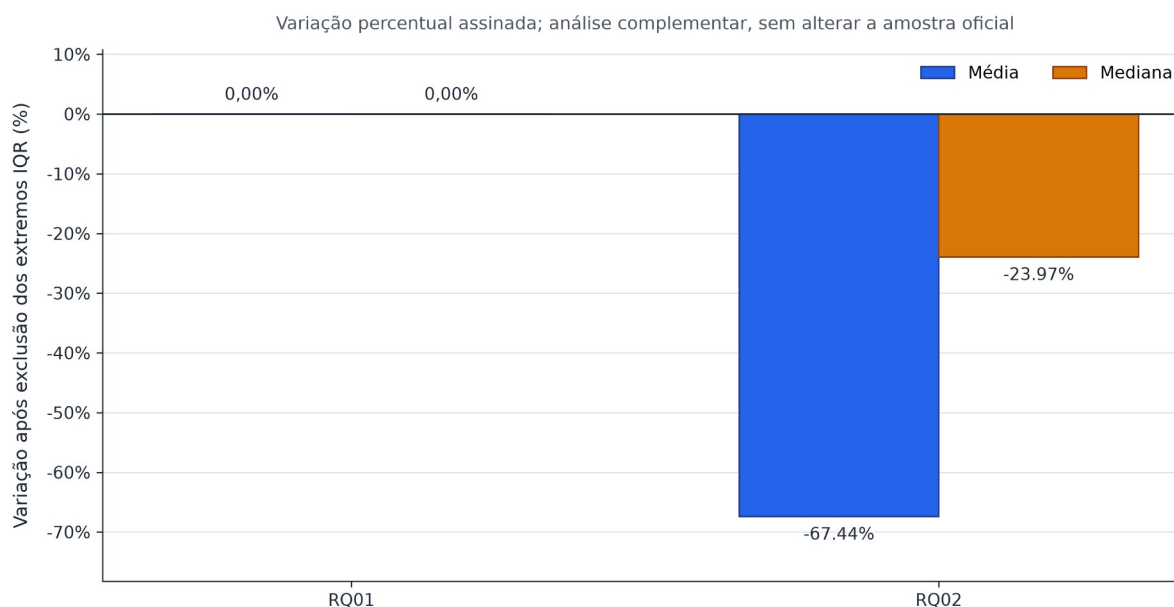


Figura 12 — Variação percentual da média e da mediana após exclusão complementar dos extremos IQR. Nenhum registro foi removido da análise oficial. Fonte: elaboração própria.

4.3 Discussão

RQ01

A hipótese informal da RQ01 foi parcialmente confirmada. A mediana de 2.821 dias — aproximadamente 7,7 anos — indica predominância de projetos maduros na amostra, mas a amplitude de 2 a 6.701 dias mostra que popularidade não está restrita a repositórios antigos. Como não foi definido um grupo de comparação nem um limiar formal de maturidade, a conclusão permanece descritiva.

RQ02

A hipótese informal da RQ02 foi parcialmente confirmada: a mediana de 765,5 PRs merged indica volume relevante no repositório típico da amostra, mas a distribuição é fortemente assimétrica. A média de 4.210,608 recebe forte influência de uma parcela minoritária de projetos com valores elevados. A métrica identifica PRs incorporadas, mas não permite afirmar que seus autores sejam necessariamente contribuidores externos.

RQ03

A hipótese informal da RQ03 foi parcialmente confirmada quanto à existência de volume relevante e forte variação entre projetos. A mediana é de 39,5 releases, enquanto 28,0% não registram releases formais. Como a operacionalização utiliza o total acumulado, o estudo não mede cadência temporal de publicação; o máximo observado de 1.000 não foi interpretado como teto da API.

RQ04

A hipótese informal da RQ04 foi sustentada descritivamente para a maioria da amostra: a mediana é de 3 dias desde o último push, e 60,2% apresentaram push nos sete dias anteriores à referência. Ainda

assim, 11,4% estavam há mais de um ano sem push. O resultado descreve recência de atualização e não demonstra, por si só, manutenção ativa ou causalidade com a popularidade.

RQ05

A hipótese informal da RQ05 foi parcialmente confirmada. Python lidera a amostra e o TIOBE, e 50,6% dos repositórios usam linguagens do top 10 adotado. A presença expressiva de TypeScript e Go, fora desse top 10, mostra que o índice e a amostra do GitHub refletem fenômenos distintos e limita uma equivalência direta entre os rankings.

RQ06

Os dados são compatíveis com a hipótese de proporção elevada de issues fechadas na amostra: a mediana é de 87,5%. Entretanto, essa razão não mede sozinha qualidade de manutenção, tempo de resposta ou capacidade dos mantenedores. Os 39 outliers mostram heterogeneidade, e os 43 casos sem issues foram corretamente mantidos como indefinidos.

RQ07

A hipótese informal da RQ07 não foi sustentada pela comparação agregada. Repositórios em linguagens fora do TIOBE top 10 apresentaram medianas maiores de PRs merged e releases e atualização mais recente do que o grupo TIOBE. O resultado é sensível à composição dos grupos, especialmente à presença de Python, TypeScript e Go, e deve ser entendido como associação descritiva, sem implicação causal.

Discussão das inovações

A análise de sensibilidade aprofundou a interpretação sem alterar a amostra oficial. A RQ01 permaneceu estável; na RQ02, a exclusão apenas complementar de 123 extremos reduziu a média de 4.210,608 para 1.371,122 (-67,44%) e a mediana de 765,5 para 582 (-23,97%), evidenciando maior resistência relativa da mediana.

A RQ07 acrescentou uma segmentação por linguagem e uma comparação com o TIOBE. Ela mostrou que a posição da linguagem no índice não se traduziu, nesta amostra, em maiores valores das métricas operacionais de atividade e atualização; ao mesmo tempo, evidenciou que a composição dos grupos limita interpretações abrangentes.

Ameaças à validade

- Estrelas como proxy de popularidade: o ranking não captura todas as dimensões de adoção, relevância ou uso.
- RQ02: PRs merged representam contribuições incorporadas, mas não identificam necessariamente autoria externa.
- RQ03: o total acumulado de GitHub Releases não mede frequência temporal, e projetos podem adotar outros mecanismos de versionamento ou distribuição.
- RQ04: pushedAt informa a recência do último push, mas não equivale, isoladamente, a manutenção ativa ou qualidade da manutenção.

- Temporalidade: os resultados retratam o estado observado no período da coleta e podem mudar com a evolução dos repositórios.
- Linguagem principal: `primaryLanguage` reduz projetos multilíngues a uma única categoria e pode retornar ausência de informação.
- Issues: a proporção de fechamento depende de como cada comunidade utiliza o sistema de Issues; projetos sem issues têm razão indefinida.
- Comparações RQ05/RQ07: a metodologia do TIOBE não equivale à frequência de linguagens em repositórios estrelados do GitHub, e as medianas agregadas são sensíveis à composição dos grupos.

5. Conclusão

O Lab01 consolidou uma coleta reproduzível de 1.000 repositórios populares, com paginação, persistência incremental, retomada, tratamento de falhas e validações automatizadas. A separação entre coleta, métricas e análise preservou o dataset bruto e produziu artefatos rastreáveis para todas as Questões de Pesquisa.

Na amostra estudada, a RQ01 mostrou predominância de repositórios maduros, com exceções recentes, e a RQ02 revelou grande heterogeneidade no volume de PRs merged. Como essa métrica não identifica a origem dos autores, ela foi interpretada apenas como contribuição aceita.

Na RQ03, a quantidade típica foi de 39,5 GitHub Releases, com 28,0% de repositórios sem releases formais; esse total acumulado não mede cadência temporal. Na RQ04, a maioria apresentou atualização recente, com mediana de 3 dias desde o último push, sem que isso seja tratado como prova de manutenção ativa.

Na RQ05, 50,6% da amostra usa linguagens do top 10 do TIOBE, com divergências relevantes como TypeScript e Go. Na RQ06, a maioria dos repositórios com valor definido apresentou alta proporção de issues fechadas, com mediana de 87,5%, sem converter a métrica em prova de qualidade da manutenção.

Como análise complementar, a RQ07 não sustentou sua hipótese agregada: o grupo fora do TIOBE top 10 apresentou medianas maiores de PRs merged e releases e menor tempo desde o último push. A análise de sensibilidade IQR, restrita às RQ01/RQ02, confirmou estabilidade da RQ01 e quantificou a influência dos valores extremos na RQ02. Nenhuma dessas análises removeu registros da amostra oficial ou estabeleceu causalidade.

As conclusões se restringem aos 1.000 repositórios e ao período observados. Devem ser consideradas as limitações das estrelas como proxy de popularidade, das métricas operacionais, da classificação por uma única linguagem e da comparação com um índice externo de metodologia distinta.

6. Referências

ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.

TUKEY, John W. Exploratory data analysis. Reading, MA: Addison-Wesley, 1977.

GITHUB. GraphQL API documentation. Disponível em: <https://docs.github.com/en/graphql>. Acesso em: 26 ago. 2026.

GITHUB. GraphQL API: Releases. Disponível em: <https://docs.github.com/en/graphql/reference/releases>. Acesso em: 26 ago. 2026.

TIOBE. TIOBE Index — August 2026. Disponível em: <https://www.tiobe.com/tiobe-index/>. Acesso em: 26 ago. 2026.

YOUTUBE. Vídeo indicado no template da disciplina. Disponível em: <https://www.youtube.com/shorts/YwnaeO95AN8>. Acesso em: 26 ago. 2026.